Leibniz FH

Wirtschaftsinformatik (B. Sc.)
Modul 20.1 (Vertiefung Entwicklung)
Sommersemester 2026

Anwendung erweiterter Softwareentwicklung

5. Laufzeitmodell

Dr. Peter Lohmann

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Leibniz FH

5.1. Motivation

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Mentales Modell

- ✓ Sprachkonzepte technisch einordnen
- ✓ Ein mentales Modell hilft, typische Probleme schon beim Entwurf zu erkennen
  - Performance
  - Memory Leaks
- ✓ Gute Entwurfsregeln werden verständlicher, wenn man ihre Laufzeitfolgen kennt
  - kleine Funktionen
  - klare Zuständigkeiten
  - bewusster Umgang mit großen Objekten

Sprachabstraktion und Maschinenmodell

- ✓ Hohe Sprachabstraktion verdeckt viele technische Details
- ✓ Implementierungsfragen wirken trotzdem direkt zurück auf Programme und Entwurfsentscheidungen
  - Speicherorganisation
  - Laufzeitverhalten
  - Effizienz
- ✓ Zentrale Frage für dieses Kapitel
  - Wie werden Namen, Aufrufe und Objekte zur Laufzeit verwaltet?

Leibniz FH

5.2. Kompilieren und Ausführen

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Compiler, Interpreter, JIT

- ✓ Compiler
  - übersetzt ein Programm vor der eigentlichen Ausführung in eine andere Form
  - Ergebnis ist ein Maschinenprogramm oder eine geeignete Zwischenrepräsentation
- ✓ Interpreter
  - führt ein Programm schrittweise unter Kontrolle eines Laufzeitsystems aus
- ✓ JIT = Just-in-Time-Compilation
  - kompiliert Teile erst während der laufenden Ausführung
  - versucht die Flexibilität eines Interpreters mit mehr Geschwindigkeit zu verbinden

Konsequenzen

- ✓ Fehler treten je nach Ausführungsmodell zu unterschiedlichen Zeitpunkten auf
  - eher vor der Ausführung
  - oder erst zur Laufzeit
- ✓ Auch die Laufzeitkosten verteilen sich unterschiedlich auf Übersetzung und Ausführung
  - mehr Vorarbeit beim Kompilieren
  - mehr Arbeit im Laufzeitsystem beim Interpretieren
- ✓ Das hat unmittelbare Auswirkungen
  - Portabilität: Kompilierter Maschinencode (oft) abhängig von OS und CPU-Typ
  - Werkzeugkette: Mehr Schritte fürs Kompilieren, komplett andere Tools
  - Optimierungsmöglichkeiten: Beim Kompilieren kann ggf. für eine spezielle System-Architektur optimiert werden

Python als Mischmodell

- ✓ Python ist technisch nicht einfach nur „rein interpretiert"
- ✓ Typischer Ablauf in *CPython*
  - Quelltext wird zunächst geparst und syntaktisch analysiert
  - daraus entsteht unter anderem ein *AST (Abstract Syntax Tree)* als interne Struktur
  - danach wird *Bytecode* für die *Python Virtual Machine* erzeugt
- ✓ Ausführung erfolgt im Laufzeitsystem
  - Namespaces, Objekte und Aufrufe bleiben dabei weiterhin dynamisch

Laufzeitobjekte in Python

- ✓ Module, Klassen und Funktionen sind in Python selbst ganz normale Laufzeitobjekte
- ✓ Diese Objekte können vom Python-Code zur Laufzeit verwendet und verändert werden
  - Namen neu binden
  - Attribute ergänzen
  - Funktionen übergeben oder zurückgeben

Leibniz FH

5.3. Speicherbereiche

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Call Stack

- ✓ Der *Stack* organisiert Funktionsaufrufe nach dem LIFO-Prinzip (Last In, First Out)
- ✓ Für jeden Funktionsaufruf entsteht ein eigener *Frame* auf dem Call Stack
- ✓ Ein solcher Frame enthält typischerweise
  - lokale Namen
  - Parameter-Werte
  - Verweis auf Parent-Frame

Heap

- ✓ Dynamisch erzeugte Objekte werden für gewöhnlich im *Heap* abgelegt
- ✓ Dazu gehören insbesondere zur Laufzeit erzeugte
  - Instanzen von vor- oder selbstdefinierten Klassen
  - Listen, Dictionaries und andere Container
  - komplexe Objektgraphen
- ✓ Der Heap wird nicht direkt vom Programmierer, sondern vom Laufzeitsystem verwaltet
  - *Automatic Memory Management*

Stack und Heap in Python

- ✓ Frames enthalten in Python (meist) nicht die Objekte selbst, sondern bloß *Referenzen*
  - Ausnahme sind Optimierungen für kleine Objekte vordefinierter Typen
- ✓ Diese Referenzen zeigen auf Objekte, die im Heap liegen
- ✓ Wichtige Folge für
  - Objektlebensdauer: Objekte können den Stack Frame, von wo aus sie erzeugt wurden, überleben!
  - Speicherlokalität: Objekte können weit weg voneinander und vom aktuellen Stack Frame liegen!
    - -> *Cache Misses*

Leibniz FH

5.4. Objekterzeugung und Lebensdauer

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Objekterzeugung

- ✓ Ein Klassenaufruf erzeugt eine neue Instanz, die im Heap abgelegt wird
- ✓ *Danach* verweisen Namen oder Attribute auf dieses Objekt und machen es erreichbar
- ✓ Begriffliche Klarstellung
  - Variablen „enthalten" in Python keine Objekte direkt
  - Sie binden Namen an Referenzen

Erreichbarkeit

- ✓ Objekte sind direkt erreichbar über verschiedene Wurzeln des laufenden Programms
  - Namen in Stack Frames
  - globale Namen
  - Attribute anderer Objekte
- ✓ Erreichbarkeit setzt sich dann indirekt über Objektstrukturen fort
  - über Listen
  - über Dictionaries
  - über verkettete Objektstrukturen / -attribute

Lebensdauer von Objekten

- ✓ Ein Objekt im Heap verschwindet nicht automatisch, nur weil eine Funktion endet
  - Im Gegensatz zum Stack, wo der Frame mit allen Namen und Referenzen sofort gelöscht wird
- ✓ Entscheidend ist nicht die syntaktische Struktur, sondern die Erreichbarkeit zur Laufzeit
- ✓ Ein Objekt lebt weiter, wenn noch mindestens eine (relevante) Referenz existiert!

Leibniz FH

5.5. Garbage Collection

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Reference Counting

- ✓ Grundidee
  - jedes Objekt zählt, wie viele Referenzen aktuell auf es zeigen
- ✓ Fällt diese Zahl auf null, kann das Objekt meist sofort freigegeben werden
- ✓ Vorteil
  - die Grundidee ist vergleichsweise einfach zu verstehen
  - viele nicht mehr benötigte Objekte werden unmittelbar freigegeben

Das Problem von Zyklen

- ✓ Reines Referenzzählen reicht in der Praxis nicht für alle Fälle aus
- ✓ Beispiel
  - Objekt A referenziert B
  - Objekt B referenziert A
- ✓ Wenn sonst niemand mehr auf beide zeigt
  - sind sie logisch unbenutzt
  - aber ihr jeweiliger Referenzzähler ist nicht null

Zyklische Garbage Collection

- ✓ Zusätzliche GC-Verfahren suchen gezielt nach unerreichbaren Gruppen von Objekten
- ✓ Entscheidend ist dabei die Erreichbarkeit vom laufenden Programm aus
  - nicht nur die lokale Zahl eingehender Referenzen
- ✓ In Python ergänzt diese zyklische Garbage Collection das einfache Reference Counting
- ✓ Trotzdem bleibt ein bewusster Umgang mit Objektreferenzen wichtig
  - große Objekte nicht unnötig festhalten
  - Referenzketten bewusst gestalten

Speicher und andere Ressourcen

- ✓ Garbage Collection verwaltet vor allem Speicher, aber nicht automatisch jede Ressource im besten Moment
- ✓ Kritische Beispiele
  - Dateien
  - Sockets
  - Datenbankverbindungen
- ✓ Deshalb bleibt explizites Ressourcenmanagement weiterhin wichtig
  - Ressourcen explizit schließen
  - with-Blöcke verwenden

Leibniz FH

5.6. Explizite Speicherverwaltung

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

C++: expliziterer Umgang mit Speicher

- ✓ In C++ kann Speicher deutlich gezielter reserviert und wieder freigegeben werden
- ✓ Klassen können Destruktoren definieren
  - für Aufräumarbeiten beim Lebensende eines Objekts
- ✓ Das bietet mehr Kontrolle über Lebensdauer und Ressourcen
  - aber auch mehr Verantwortung

Typische Fehler expliziter Verwaltung

- ✓ Memory Leaks
  - Speicher wird reserviert, aber später nicht korrekt wieder freigegeben
- ✓ Zu frühe Freigabe
  - Referenzen oder Zeiger zeigen danach auf ungültigen Speicher
- ✓ Praktische Folgen
  - undefiniertes Verhalten
  - Abstürze und Segmentation Faults

Vergleich der Ansätze

- ✓ Automatische Speicherverwaltung
  - ist für Anwendungsentwicklung meist bequemer
  - ist im Alltag oft deutlich weniger fehleranfällig
- ✓ Explizite Speicherverwaltung
  - bietet mehr Kontrolle über Lebensdauer, Speicherverbrauch und Kosten
  - erzeugt aber auch ein höheres Fehlerrisiko
- ✓ Für Python wichtig
  - Speicherverwaltung ist automatisiert
  - Ressourcenmanagement und Referenzdisziplin bleiben trotzdem relevant